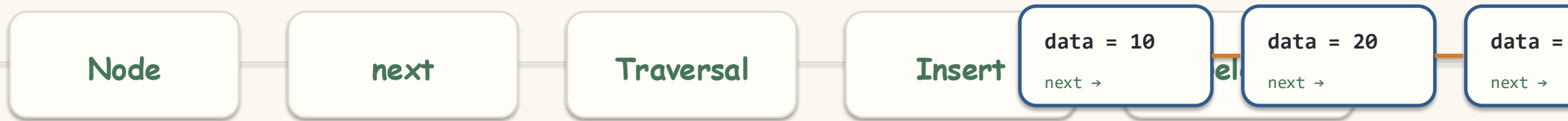


LINKED LIST FUNDAMENTALS

From array thinking → pointer thinking

A visual, interview-ready foundation for singly linked lists.

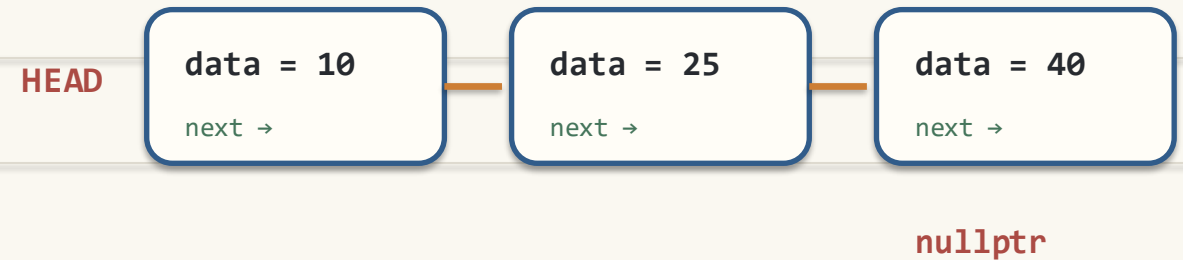


Goal → understand the structure before memorizing code.

1. What is a Linked List?

Mental model

- A linear data structure made of separate nodes.
- Each node stores data + a link to the next node.
- Nodes do NOT need contiguous memory.
- HEAD points to the first node; the last node points to nullptr.



The list is connected by links, not by adjacent addresses.

2. Why Linked List vs Array?

Feature	Array	Linked List
Memory	Contiguous	Non-contiguous
Random access	$O(1)$	$O(n)$
Insert/delete at front	$O(n)$	$O(1)^*$
Resize	Fixed / costly	Dynamic
Extra memory	Low	Pointer overhead

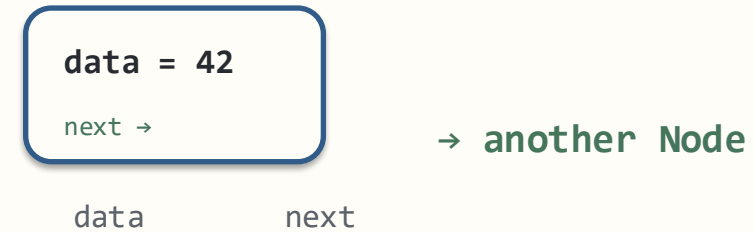
* If the node/location is already known. Finding that position can still cost $O(n)$.

3. Node Structure + Pointers

C++ node

```
struct Node {  
    int data;  
    Node* next;  
  
    Node(int value) {  
        data = value;  
        next = nullptr;  
    }  
};
```

Picture it

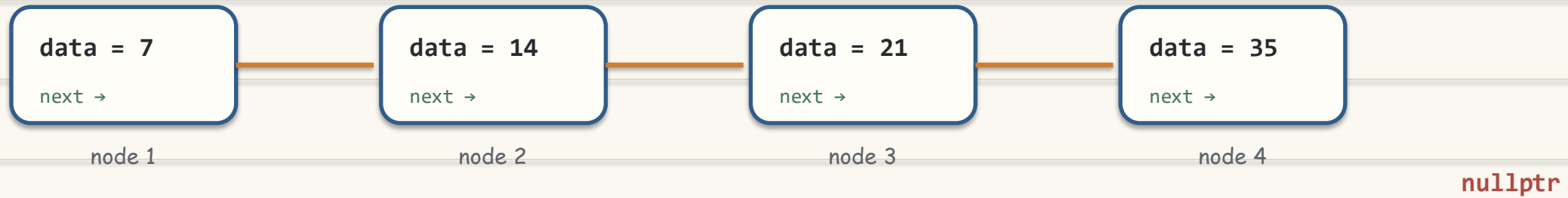


Pointer = stores the address of another object.

4. Singly Linked List

One direction →

HEAD



Core rule

You can move from HEAD → next → next → ... but not backward.

5. Creating + Traversing

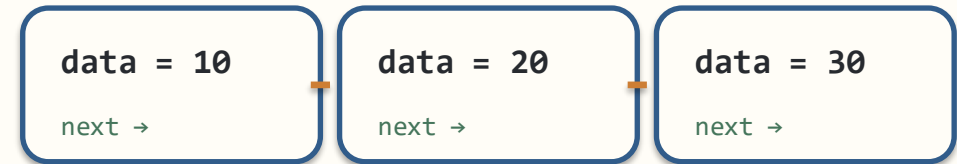
Create

```
Node* head = new Node(10);  
head->next = new Node(20);  
head->next->next = new Node(30);
```

Traverse

```
Node* temp = head;  
while (temp != nullptr) {  
    cout << temp->data << " ";  
    temp = temp->next;  
}
```

Traversal



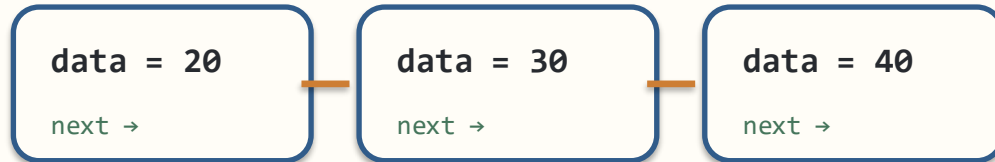
temp

Move temp one link at a time until nullptr.

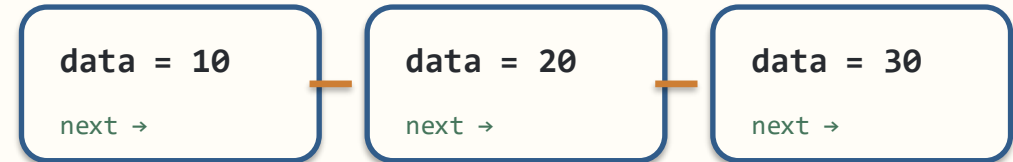
6. Insertion — Beginning

Before

HEAD



Insert 10

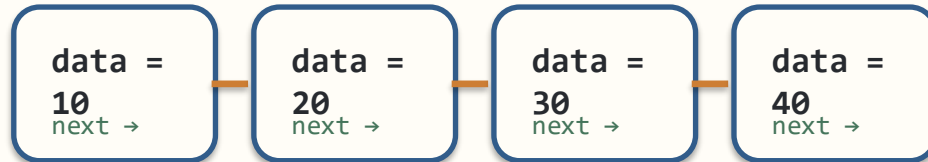


```
newNode->next = head;  
head = newNode;
```

Time: $O(1)$

7. Insertion — Middle / End

Middle: insert 25



Find node 20 → connect new node →
reconnect 30.

```
newNode->next = curr->next;  
curr->next = newNode;
```

End

- Traverse until `temp->next == nullptr`.
- Attach new node to the last node.
- If you maintain a tail pointer, insertion can be $O(1)$.
- Without tail: finding the end is $O(n)$.

8. Deletion Basics

Three common cases

Delete first

```
head = head->next;
```

$O(1)$

Delete middle

```
prev->next = curr->next;
```

$O(n)$ to find

Delete last

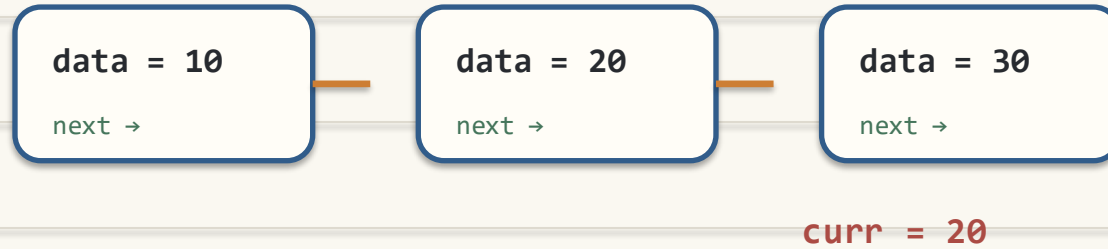
```
prev->next = nullptr;
```

$O(n)$ to find

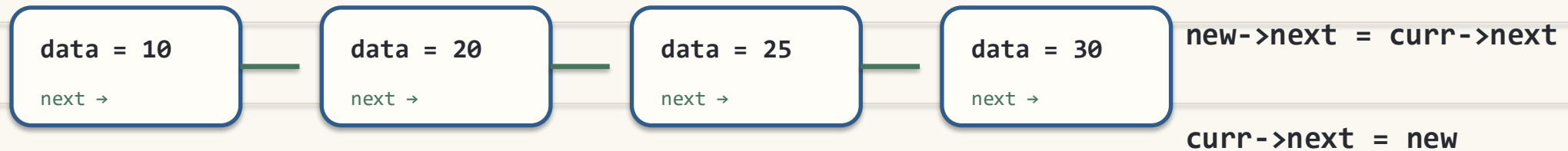
Always handle nullptr / single-node cases before changing links.

9. Dry Run — Insert 25 After 20

Step 1



Step 2



Final chain: 10 → 20 → 25 → 30

10. Complete C++ Implementation

Singly Linked List — core operations

```
struct Node {  
    int data;  
    Node* next;  
    Node(int x) : data(x), next(nullptr) {}  
};  
  
void print(Node* head) {  
    while (head) {  
        cout << head->data << " ";  
        head = head->next;  
    }  
}  
  
void pushFront(Node*& head, int x) {  
    Node* n = new Node(x);  
    n->next = head;  
    head = n;  
}  
  
void deleteFront(Node*& head) {  
    if (!head) return;  
    Node* t = head;  
    head = head->next;  
    delete t;  
}
```

11. Complexity Cheat Sheet

Operation	Time	Extra Space
Traverse	$O(n)$	$O(1)$
Search by value	$O(n)$	$O(1)$
Insert at head	$O(1)$	$O(1)$
Delete head	$O(1)$	$O(1)$
Insert at known node	$O(1)$	$O(1)$
Insert at position	$O(n)$	$O(1)$

Space note: the list itself uses $O(n)$ nodes; the table shows extra auxiliary space.

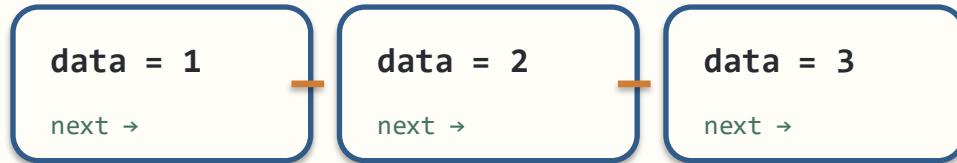
12. Common Mistakes

- ❌ Losing the head pointer while traversing
- ❌ Dereferencing nullptr
- ❌ Forgetting to update links in the correct order
- ❌ Memory leak after deleting a node
- ❌ Treating a linked list like an array (expecting $O(1)$ indexing)
- ❌ Ignoring empty-list and one-node edge cases

Rule of thumb: draw the arrows first, then write the code.

13. Interview Pattern — Pointer Rewiring

Before



Thinking pattern

- Keep a reference to what comes next.
- Change one pointer at a time.
- Move the working pointer forward.
- This idea powers reversal, merging and many list problems.

prev ← curr → next

14. Interview / LeetCode Practice

LC 206

Reverse Linked List

Pointer rewiring + prev/curr

LC 21

Merge Two Sorted Lists

Two pointers + linking

LC 141

Linked List Cycle

Fast & slow pointers

Practice order → understand traversal → pointer rewiring → two-pointer patterns.

15. Real-World Use Case

Where the idea appears

- Browser history / navigation concepts
- Music or media playlists
- Memory allocators and free lists
- Undo/redo structures (often doubly linked)
- Graph adjacency structures can use linked nodes

Why it matters

When elements are frequently inserted/removed and random indexing is less important, link-based structures can be useful.

Design question → "Do I need fast indexing, or fast local insertion/deletion?"

16. Key Takeaways

- A linked list = nodes connected through pointers.
- HEAD is the entry point; last->next is nullptr.
- Traversal is $O(n)$; indexing is not $O(1)$.
- Insertion/deletion can be $O(1)$ when the relevant node/link is already known.
- Always protect against nullptr and edge cases.
- Master pointer rewiring — it unlocks many interview problems.

Think in links, not indexes.

17. DAY 42 PREVIEW — LINKED LIST OPERATIONS

Next: go deeper into manipulation.

Day 42

- Insert & delete at specific positions
- Reverse a linked list — iterative approach
- Pointer movement: prev / curr / next
- Dry runs + edge cases
- Interview-focused problems

Preview → Reverse Linked List is the key pointer pattern.